

Food Store

Sistema de Gestión de Pedidos de Comida

Materia: Programación 2

Trabajo Práctico Integrador

Integrantes del grupo:

Gonzales Patricio

Zuvialde Ignacio

Azcurra Matias

Año 2026

Índice

1. Marco Teórico	3
2. Decisiones Técnicas y Arquitectura	4
2.1 Estructura de paquetes	4
2.2 Diagrama de clases (descripción)	5
2.3 Capturas del sistema funcionando	6
3. Dificultades y Soluciones	7
4. Bibliografía / Webgrafía	8
5. Links del proyecto	8

1. Marco Teórico

Java 21

Java es un lenguaje de programación orientado a objetos, multiplataforma y de propósito general. En su versión 21 incorpora características modernas como switch expressions, records y sealed classes, además de mantener su sólida base de Programación Orientada a Objetos (POO). Es ampliamente utilizado en aplicaciones empresariales, de escritorio y de consola.

Programación Orientada a Objetos (POO)

La POO es un paradigma de programación basado en la creación de objetos que combinan datos (atributos) y comportamientos (métodos). Los cuatro pilares fundamentales son:

- **Encapsulamiento:** los atributos son privados y se accede a ellos mediante getters y setters.
- **Herencia:** las clases hijas (Categoria, Producto, Usuario, Pedido) heredan de la clase abstracta Base.
- **Polimorfismo:** la interfaz Calculable permite que Pedido implemente su propio método calcularTotal().
- **Abstracción:** la clase Base define la estructura común sin ser instanciable directamente.

Colecciones de Java

Las Colecciones son estructuras de datos dinámicas proporcionadas por el Java Collections Framework. En este proyecto se utilizó principalmente ArrayList para almacenar entidades en memoria durante la ejecución. A diferencia de los arrays estáticos, los ArrayList permiten agregar y eliminar elementos de forma dinámica, lo que facilita la gestión del ciclo de vida de los objetos (alta, baja lógica, modificación).

Manejo de Excepciones

Java permite crear excepciones propias extendiendo la clase Exception. En este proyecto se implementaron tres excepciones personalizadas para representar situaciones específicas del dominio:

- **EntidadNoEncontradaException:** se lanza cuando se busca un elemento por ID y no existe o está eliminado.
- **StockInvalidoException:** se lanza cuando la cantidad de un detalle de pedido es inválida o supera el stock.
- **ValidacionException:** se lanza cuando los datos ingresados no cumplen las reglas de negocio (campos vacíos, mail duplicado, precio negativo, etc.).

Interfaz Calculable

Una interfaz en Java define un contrato de comportamiento. La interfaz Calculable define el método calcularTotal(), que la clase Pedido implementa para sumar los subtotales de todos sus detalles. Esto garantiza que cualquier entidad que implemente Calculable deberá tener su propia lógica de cálculo.

2. Decisiones Técnicas y Arquitectura

2.1 Estructura de paquetes

El proyecto se organizó en capas bien definidas para separar responsabilidades y facilitar el mantenimiento del código:

```
src/integrado/prog2/
■■■ Main.java           → Punto de entrada, instancia servicios y menús
■■■ entities/           → Clases del modelo (UML)
■   ■■■ Base.java       → Clase abstracta con id, eliminado, createdAt
■   ■■■ Calculable.java → Interfaz con calcularTotal()
■   ■■■ Categoria.java
■   ■■■ Producto.java
■   ■■■ Usuario.java
■   ■■■ Pedido.java     → Implementa Calculable
■   ■■■ DetallePedido.java
■■■ enums/              → Tipos enumerados
■   ■■■ Rol.java         → ADMIN, USUARIO
■   ■■■ Estado.java      → PENDIENTE, CONFIRMADO, TERMINADO, CANCELADO
■   ■■■ FormaPago.java   → TARJETA, TRANSFERENCIA, EFECTIVO
■■■ exception/          → Excepciones propias
■   ■■■ EntidadNoEncontradaException.java
■   ■■■ StockInvalidoException.java
■   ■■■ ValidacionException.java
■■■ service/            → Lógica de negocio, manejo de colecciones
■   ■■■ CategoriaService.java
■   ■■■ ProductoService.java
■   ■■■ UsuarioService.java
■   ■■■ PedidoService.java
■■■ ui/                 → Menús de consola (Scanner)
■   ■■■ CategoriaMenu.java
■   ■■■ ProductoMenu.java
■   ■■■ UsuarioMenu.java
■   ■■■ PedidoMenu.java
```

2.2 Decisiones de diseño tomadas

Capa de servicios separada de la UI:

Cada servicio (CategoriaService, ProductoService, etc.) contiene toda la lógica de negocio y manejo de colecciones. Los menús solo reciben datos del usuario y llaman a los servicios, sin contener reglas de negocio.

Soft delete:

Ninguna entidad se elimina físicamente de la colección. En cambio, se marca el campo eliminado = true. Esto preserva el historial y evita inconsistencias (por ejemplo, un pedido puede seguir referenciando un usuario eliminado).

Generación de IDs automática:

Cada servicio mantiene un contador idCounter que se incrementa con cada nueva entidad creada, simulando el comportamiento de un ID autoincremental de base de datos.

Cancelación de pedido ante error:

Si ocurre un `StockInvalidoException` al agregar un detalle durante la creación de un pedido, el pedido completo se cancela y se elimina de la colección para no dejar datos inconsistentes.

Validación de unicidad de mail:

El `UsuarioService` recorre la colección antes de crear o editar un usuario para asegurarse de que el mail no exista ya en otro registro activo.

2.3 Ejemplo de uso del sistema (flujo principal)

A continuación se describe el flujo típico de uso del sistema desde la consola:

1. Se inicia el programa y aparece el menú principal con las 4 opciones.
2. El operador crea una Categoría (ej: 'Pizzas').
3. El operador crea un Producto asociado a esa categoría (ej: 'Pizza Muzzarella', precio \$1500, stock 10).
4. El operador crea un Usuario (ej: 'Juan Pérez', mail `juan@mail.com`).
5. El operador crea un Pedido: selecciona al usuario, elige forma de pago, agrega productos con cantidades.
6. El sistema calcula el total usando `calcularTotal()` de la interfaz `Calculable`.
7. El operador puede actualizar el estado del pedido (PENDIENTE → CONFIRMADO → TERMINADO).
8. Al finalizar, puede eliminar lógicamente cualquier entidad.

Nota: Las capturas de pantalla del sistema en funcionamiento se mostrarán en el video demostrativo.

3. Dificultades y Soluciones

Dificultad 1: Colisión de nombres entre clases

Durante el desarrollo notamos que varios conceptos tenían nombres similares (por ejemplo, manejar los IDs de DetallePedido dentro de Pedido). Resolvimos centralizar el contador de IDs de detalles como atributo estático dentro de la clase Pedido, evitando así depender de un servicio externo para esta responsabilidad.

Dificultad 2: Manejo correcto del soft delete en listados

Al principio los listados mostraban también los elementos eliminados. La solución fue filtrar en cada método listar() del servicio, recorriendo la colección y agregando solo los elementos donde eliminado == false.

Dificultad 3: Cancelación de pedido ante error en detalle

Surgió la duda de cómo manejar un error al agregar un detalle a un pedido que ya había sido agregado a la colección. La solución fue capturar el StockInvalidoException en el menú y llamar a pedidoService.cancelarPedido(pedido) para removerlo de la lista antes de que quede en estado inconsistente.

Dificultad 4: Validación de entradas del usuario en consola

Los menús deben manejar entradas inválidas (letras donde se espera número, campos vacíos, opciones fuera de rango). Resolvimos envolver las lecturas numéricas en bloques try-catch con NumberFormatException, y en los servicios validamos cadenas vacías lanzando ValidacionException.

Dificultad 5: Relación entre Pedido y DetallePedido

Fue necesario entender que DetallePedido es una composición dentro de Pedido: los detalles no tienen vida propia fuera del pedido. Por eso los métodos addDetallePedido, findDetallePedidoByProducto y deleteDetallePedidoByProducto viven directamente en la clase Pedido, tal como lo indica el UML.

4. Bibliografía / Webgrafía

- Documentación oficial de Java 21 – Oracle: <https://docs.oracle.com/en/java/javase/21/>
- Java Collections Framework – Oracle: <https://docs.oracle.com/javase/8/docs/technotes/guides/collections/>
- Programación Orientada a Objetos en Java – Tutoriales ofrecidos por la cátedra
- Manejo de Excepciones en Java: <https://docs.oracle.com/javase/tutorial/essential/exceptions/>
- Interfaces en Java: <https://docs.oracle.com/javase/tutorial/java/landl/createinterface.html>
- Enums en Java: <https://docs.oracle.com/javase/tutorial/java/javaOO/enum.html>

5. Links del proyecto

- Repositorio GitHub: <https://github.com/matiiazc/Food-Store>